

Translation Table

Three floppies of Indiana Jones and the Last Crusade: The Graphic Adventure carry no copy protection at all. The lock was printed on a card you read through red plastic — and the routine that checks it also regenerates it, so the card is recoverable from the bytecode.

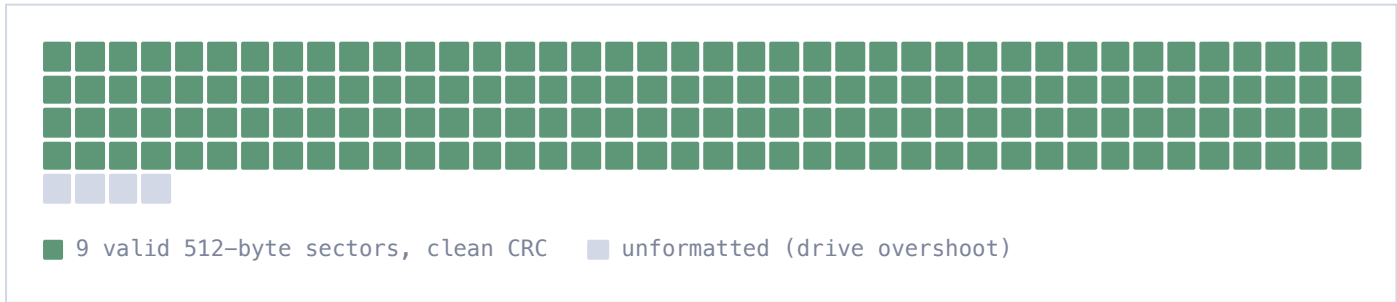
Source 3 × Pasti/STX images Engine SCUMM v3 (indy3) Release Kixx, 720 KB GEMDOS

The media is clean. All 480 formatted tracks read as ordinary, byte-perfect MFM.

No fuzzy bits, no deliberate CRC errors, no duplicate or out-of-range sector IDs, no long tracks, no extra sectors. These disks could be duplicated with any ordinary copier — and that was fine, because the thing you couldn’t duplicate came in the box. The check is 552 bytes of SCUMM bytecode in room 92, and it is printed in full below.

What a protected disk would have looked like

The Pasti (`.stx`) format exists precisely to preserve copy protection: it stores per-sector flux timing, fuzzy-bit masks, FDC status flags and a full MFM track image, so that weak bits and malformed sectors survive a dump. All of that machinery is present in these files and all of it is empty.



All 164 track records of disk 1, side 0 and side 1 interleaved. Tracks 0–79 are uniformly standard; tracks 80–81 hold no sectors at all — the dumping drive simply stepped past the end of the formatted area. Disks 2 and 3 are identical in structure.

PROTECTION TECHNIQUE	DETECTABLE AS	FOUND
Weak / fuzzy bits	non-zero fuzzy mask	✓ 0 bytes, all tracks
Deliberate CRC errors	FDC status flags set	✓ 0 sectors
Duplicate / phantom sector IDs	repeated ID field	✓ IDs 1–9, once each
Non-standard sector size	size code ≠ 2	✓ all 512 bytes
Long / short tracks	outlier track length	✓ within normal band
Data past track 79	sectors on track 80+	✓ unformatted
Direct FDC access in code	Flopfmt / write-track buffer	✓ none in INDY.PRG

Seven ways an Atari ST title of this era could have protected itself. None of them are used.

— EVIDENCE · THE FILESYSTEM

An ordinary GEMDOS floppy

Under the Pasti wrapper each disk is a textbook 720 KB Atari floppy: 80 tracks, 2 sides, 9 sectors, 2 sectors per cluster, two 5-sector FATs, a 112-entry root directory starting at sector 11. There is no volume label, and the boot sector’s OEM field reads `FAST` — the signature of a bulk duplication tool, with plain `E5` filler after the BPB and a checksum of `0x1235`, one deliberate step away from the `0x1234` that would make it bootable. Files mount and extract with no tricks:

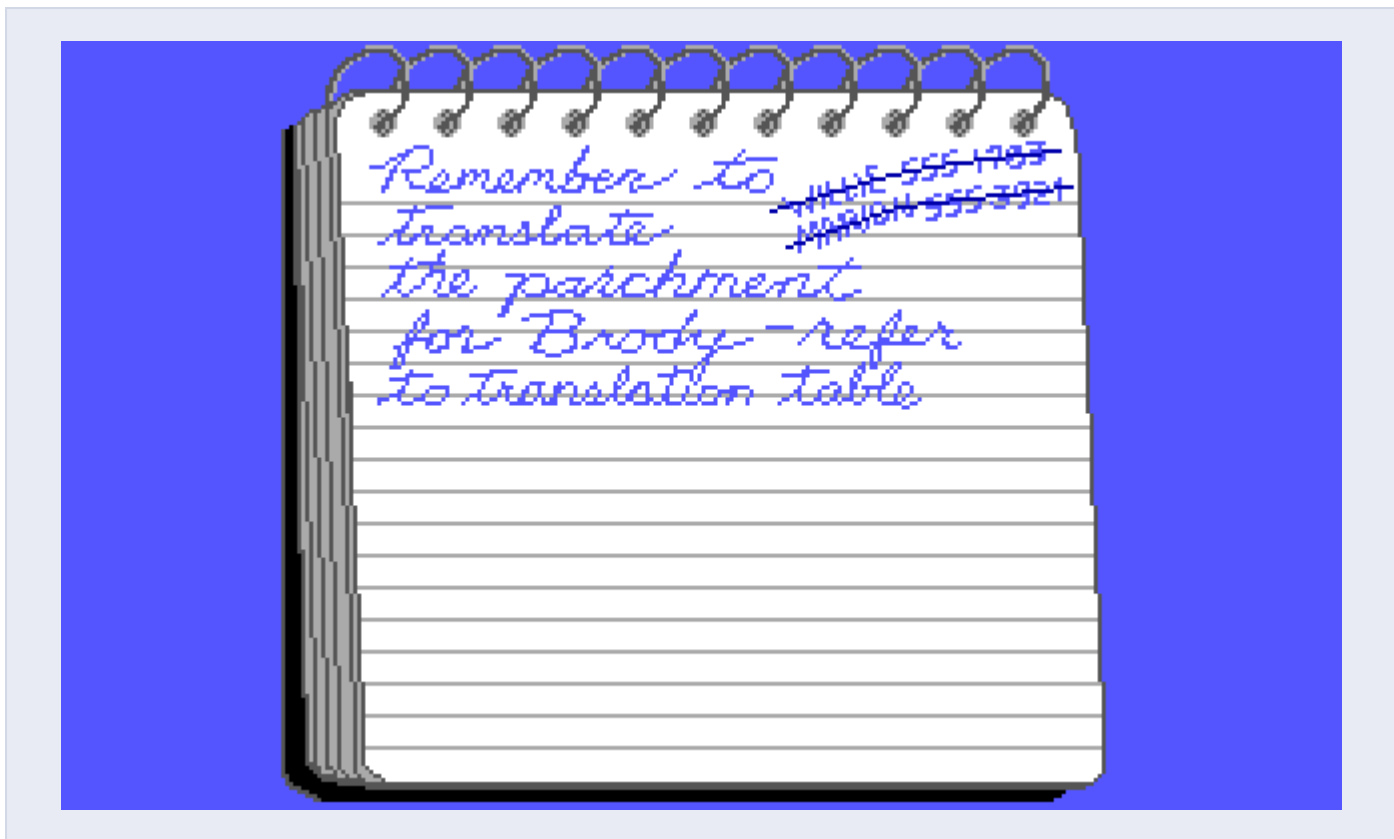
kixx1.st bps=512 spc=2 res=1 nfat=2 ndir=112 tot=1440 spf=5 root@11 data@18					
DISK1.ID	0	DISK2.ID	0		
00.LFL	5361	01.LFL	44276	02.LFL	11728
03.LFL	21214	04.LFL	4604	06.LFL	18289
07.LFL	12721	14.LFL	16500	45.LFL	23875
90.LFL	4705	91.LFL	4432	92.LFL	16558
98.LFL	2042	99.LFL	2042	DESKTOP.INF	478
INDY.PRG	104287				

Abridged root directory of disk 1. `INDY.PRG` is the SCUMM v3 interpreter; the `NN.LFL` files are the game rooms, XOR-encrypted with `0xFF`. `98.LFL` and `99.LFL` are the only unencrypted resources — they are the character sets.

— EVIDENCE · THE GATE

Room 92 asks for something you own on paper

Decrypting the room files and decoding their EGA bitmaps turns up the check immediately. Room 92 on disk 1 is a notepad, and the handwriting on it is the entire protection scheme stated out loud.



Room 92, background layer, decoded from `92.LFL` and rendered at 2x. “Remember to translate the parchment for Brody — refer to translation table”, scrawled over two struck-out phone numbers for Julie and Marion.

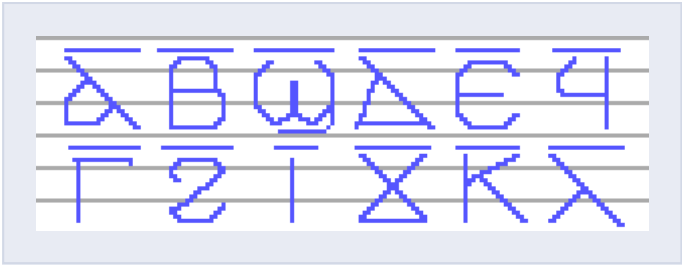
The room’s scripts contain the rest of the exchange. Four strings carry the whole mechanism:

```
14773: That doesn't seem to be right.
14819: No, that's not right either.
14863: Oh well^
14873: I can't do it without my Translation Table.
14974: Check Section
14992: , Row
15002: , Column
```

Text extracted from `92.LFL` at the listed byte offsets. “I can’t do it without my Translation Table”, followed by a **Section / Row / Column** coordinate. That is a lookup into a printed grid — the classic manual-check protection, dressed up as a story beat.

The alphabet it asks about

Two 152×48 object bitmaps in the same room hold the glyphs. The first is the plain alphabet; the second is the cipher script the player has to match against the printed table.



Left: the reference alphabet. Right: the cipher forms. Both decoded from object image blocks inside 92.LFL and rendered at 3x. Twelve symbols each — the game shows you one and the printed table tells you what it means.

THE CHECK

Reading the bytecode

The room’s scripts are ordinary SCUMM v3 bytecode, so the check can be read directly. Entering room 92 runs local script 202, which draws three random numbers and stores them in globals 103, 104 and 105:

NAME USED HERE	SCUMM VARIABLE	MEANING
row	VAR103	1–26, shown to the player as a letter A–Z
column	VAR104	1–7
section	VAR105	1–4; also picks which glyph strip is on screen
expect1–4	VAR99–VAR102	the four symbols the card should give, in the order asked
pick	local0	the symbol the player just chose
entryPos	VAR106	which of the four is being entered; 5 = all in
wrongFlag	bit1508	sticky — set by any mismatch, cleared only on reset
strikes	VAR107	failed attempts; selects the refusal message
sceneDone	VAR57	scene finished — set on success <i>and</i> on the third strike
scratch	VAR53	general scratch: the row letter, then a string id
t	local0	the accumulator in the derivation

The game has no symbol table, so these names are mine, assigned from observed behaviour. Every listing below uses them; the raw variable numbers are here and in the full disassembly. VAR41, VAR20 and VAR21 also appear, seeding the random stir — I did not identify what they hold elsewhere in the game, so they keep their numbers.

```
14626 row      = getRandomNr(25) + 1      ; 1..26
14639 column   = getRandomNr(6)  + 1      ; 1..7
14652 section  = getRandomNr(3)  + 1      ; 1..4
...

```

```

14927 scratch = row + 64 ; row number -> letter
14938 string[9] = "#####"
14947 string[9][0] = scratch ; 1..26 becomes 'A'..'Z'
14958 scratch = 9 ; reused, now a string id
14963 print(254): center pos(165,113)
      text="Check Section \4(section), Row \7(scratch), Column \4(column)."

```

The prompt is assembled at run time. `\4` prints a variable as a number, `\7` prints a string — so the row is shown as a letter and the section and column as digits. The player is told a coordinate and has to read the answer off the card.

Four symbols are then entered one at a time. `entryPos` counts the position, `pick` is the symbol the player chose, and `wrongFlag` is the sticky failure flag:

```

15026 sum = expect1 + expect2 + expect3 + expect4
15045 if (sum == 0) wrongFlag = 1 ; anti-tamper guard, see below

15057 if (entryPos == 1) { if (expect1 != pick) wrongFlag = 1 } goto next
15079 if (entryPos == 2) { if (expect2 != pick) wrongFlag = 1 } goto next
15101 if (entryPos == 3) { if (expect3 != pick) wrongFlag = 1 } goto next
15123 if (entryPos == 4) { if (expect4 != pick) wrongFlag = 1 }
15145 next: entryPos++
15148 if (entryPos == 5) { ; all four entered
15155     if (wrongFlag != 0) { ; at least one was wrong
15160         strikes++
15163         startScript(203, [0]) ; show the refusal message
15169         waitForMessage
15170         if (strikes == 3) sceneDone = 1 ; give up, move on anyway
15185         else { entryPos = 1; wrongFlag = 0 } ; otherwise reset and retry
        } else {
15198         sceneDone = 1 ; answered correctly
        } }

```

expect1–4 hold the symbols the card should give. The flag is sticky and silent: nothing is reported per symbol, so the verdict only lands once all four are in. **strikes** counts failures and selects the three refusals in order — “That doesn’t seem to be right”, “No, that’s not right either”, and “Oh well — I can’t do it without my Translation Table.”

sceneDone (VAR57) is not a pass flag. It is set on a correct answer *and* on the third strike, so it marks the scene as finished rather than passed: after three failures the game simply stops asking and play continues. Before that, retries are unlimited — only the strike counter climbs.

The guard at 15045 is an anti-tamper check rather than a gameplay case: `expect3`’s range is 3–11, so a real key can never sum to zero. It fires only if the derivation never ran.

Where the expected answers come from

This is the interesting part. `expect1–4` are not read from a table on disk — there isn’t one. They are *computed* from the coordinate by a stretch of deliberately convoluted integer arithmetic at `0x3B66`, using nothing but add, subtract and compare. Rewritten in order:

```

row (VAR103) 1..26      column (VAR104) 1..7      section (VAR105) 1..4

t = column + 17 * section      ; the 17*section is an unrolled loop
t += 2 * row
if column >= 3: t -= 2
t += row
if row < 5: t += 3
t += 12
do { t -= 9 } while (t >= 12)      expect3 = t

t += column + row + 19
if row > 7: t -= 5
t += 7
do { t -= 7 } while (t >= 8)
t += 19
if t <= expect3: t += expect3
do { t -= 12 } while (t > 11)      expect4 = t

t += 3 * column + row + 14
if row < 10: t += row
do { t -= 5 } while (t >= 10)
if t != expect4: t += expect3
t += 23
do { t -= 12 } while (t >= 12)      expect2 = t

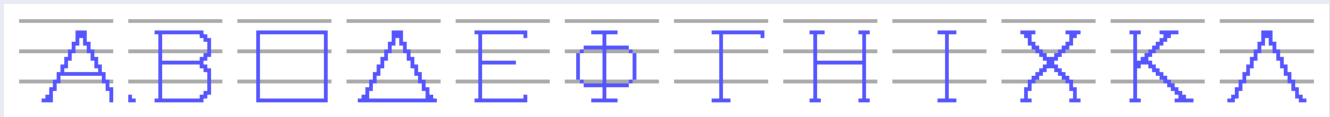
t += 3 * column + row + 13
if column > 7: t += 6      ; dead code – column never exceeds 7
do { t -= 8 } while (t >= 17)
if t > expect2: t += expect4
t += 14
do { t -= 12 } while (t > 11)      expect1 = t

```

333 bytes of bytecode, `0x3B66` to `0x3CB2`. Every result lands in 0–11 — exactly the twelve glyphs on screen, which is a satisfying cross-check on the decoding. The chaining is intentional: each symbol folds in the previous one, so you cannot work out one cell of the card without computing the others. Note the distribution is uneven — `expect3` only ever takes 9 of the 12 values and `expect4` only 7.

Which glyph is which

The twelve object bitmaps slice cleanly in reading order, so index 0–11 maps onto a concrete alphabet:



Indices 0–11 left to right: `A B □ Δ E Φ Γ H I X K Λ`.

The card, reconstructed

Because the answers are computed rather than stored, the routine *is* the printed card. Running it over every coordinate regenerates the whole Translation Table — 4 sections × 26 rows × 7 columns = 728 cells, four symbols each. Section 1 in full:

SECTION 1 — SYMBOLS LISTED IN THE ORDER THE GAME ASKS FOR THEM							
	COL 1	COL 2	COL 3	COL 4	COL 5	COL 6	COL 7
A	AEXX	KΦKΛ	K□XΛ	HΓKB	KHΛI	ΛXΔI	BKEK
B	AXΔΛ	EIEB	AKΔB	IHEI	□AΦK	□ΛΓA	EAH□
C	ΑΛΓI	□AHK	Λ□ΓK	ΔBHA	B□I□	AΔXX	KEKΛ
D	ΔBXA	B□K□	ΓEX□	XΦKX	ΔΓΛΛ	AΙΔΛ	ΦAEB
E	AEXB	BHKI	Λ□XI	XΔKK	ΓHΛA	ΔXΔA	ΛKE□
F	BXΔI	ΛKEK	ΛHΔK	ΑΛΕA	□AΦ□	BBΓX	Λ□HΛ
G	ΑΛΓA	□AH□	H□Γ□	ΔIHX	EEIΛ	AΔXB	ΛEKI
H	□ΦXΛ	ΓEKB	ΓBXB	X□KI	ΔΔΛK	ΛKΔK	BXEΛ
I	AΙΔB	ΛXEI	ΑΛΔI	KAEK	ΓΛΦA	IAG□	ΛBHX
J	KAGK	ΓΛHA	ΛBΓA	Φ□H□	IΔIX	□EXΛ	ΓΔKB
K	EBX□	Δ□KX	AEXX	KΦKΛ	□EΛB	EΛΔB	ΛAEI
L	BKΔX	ΑΛΕΛ	ΛΙΔΛ	EAEB	HΒΦI	B□ΓK	ΦBHA
M	HΛΓB	□AHI	Λ□ΓI	XΔHK	B□IA	HΔX□	KEKX
N	E□XK	ΦΓKA	ΦΔXA	ΔEK□	□ΦΛX	ΛHΔX	ΦIEΛ
O	ΓXΔA	EKE□	EHΔ□	ΔIEX	BXΦΛ	ΓBΓB	□HHI
P	ΔAGX	BBHΛ	ΦKΓΛ	□□HB	XHII	ΛEXK	ΔΔKA
Q	BBXB	A□KI	AHXI	ΔΦKK	ΛEΛA	BΛΔA	ΔAE□
R	HHΔI	ELEK	EΙΔK	BAEA	HΒΦ□	IΙΓX	EΔHΛ
S	EΛΓA	ΓAH□	Δ□Γ□	ΛIHX	AEIΛ	EΔXB	HEKI

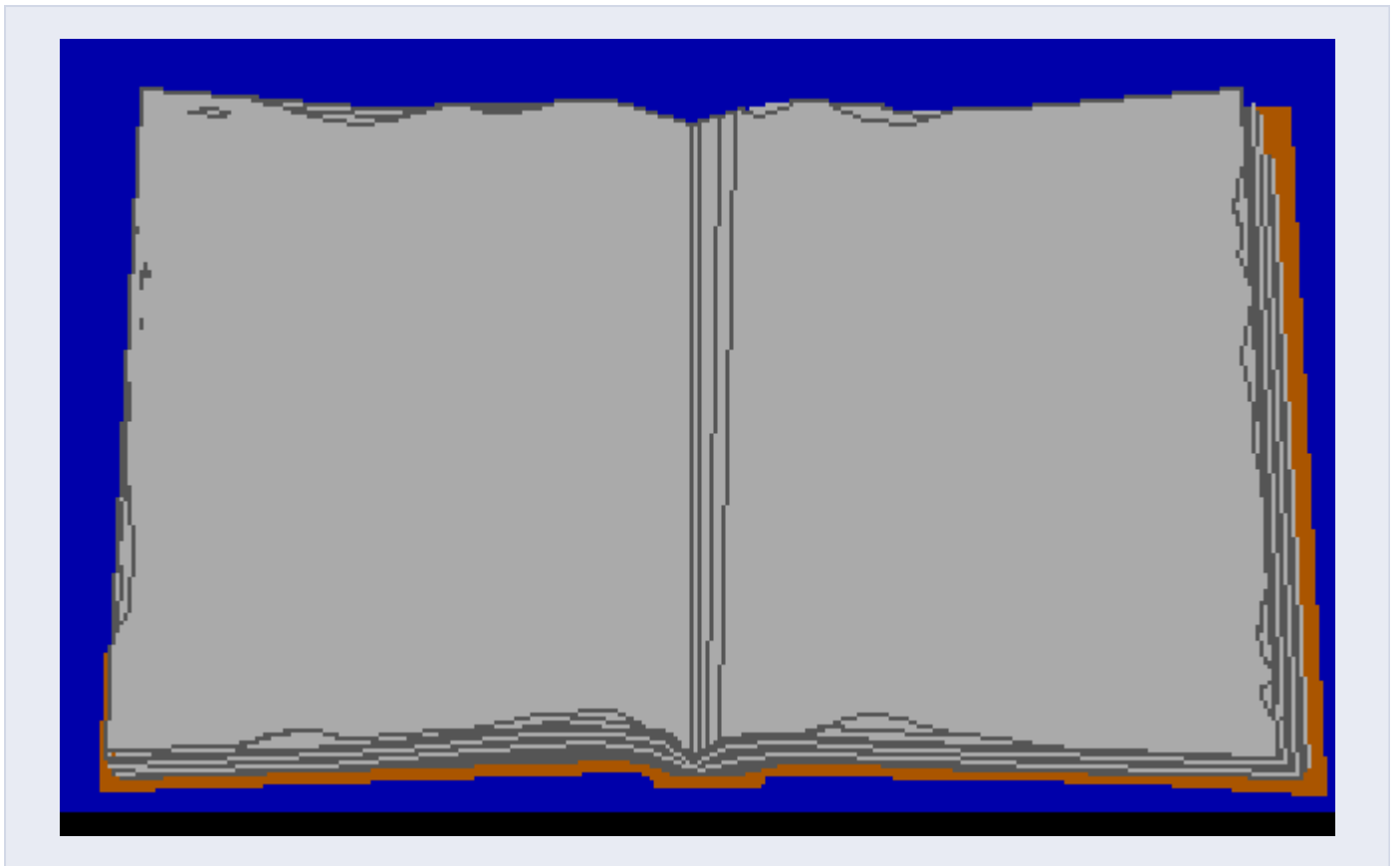
	COL 1	COL 2	COL 3	COL 4	COL 5	COL 6	COL 7
T	B□XX	ΛΔKΛ	EΦXΛ	AEKB	ΛΦΛI	IΗΔI	□IEK
U	ΦΛΔΛ	AKEB	BHΔB	AIEI	KXΦK	ΔBΓA	B□H□
V	AAΓI	KBHK	□KΓK	Λ□HA	ΦΔI□	IEXX	□ΦKΛ
W	ΓBXA	E□K□	BEX□	AΦKX	KΓΛΛ	ΔIΔΛ	AAEB
X	AKΔ□	BΛEX	BIDX	ΛXEΛ	EBΦB	H□ΓI	BΔHK
Y	ΛBΓΛ	ΔAHB	A□ΓB	ΛΔHI	XEIK	BΔXA	HEK□
Z	K□XI	EΔKK	BΦXK	ΦEKA	ΔΦΛ□	IΗΔ□	ΛIEX

So when the game asks for *Section 1, Row C, Column 4*, the card said **ΔBHA** — and that is what you clicked. Sections 2–4 come out of the same function.

ELSEWHERE IN THE GAME

The Grail Diary is a different puzzle

Room 90 looks like a second protection screen and isn't one. It is Henry Jones's diary, opened as a full-screen prop, and it exists to *give* the player an answer rather than demand one. Worth separating out, because the resemblance is misleading.



Room 90 background at 2x. The pages are drawn empty; everything else is composited at run time.

Twelve objects, **913–924**, hold the page art. The entry script hides all twelve, then draws six — three on each page:

```
4398 local0 = 913 .. 924: setState(local0, 0)      ; hide all twelve
4417 scratch = 913 + correct1 ; drawObject(scratch, 5, 6)    ; left page
4435 scratch = 917 + correct2 ; drawObject(scratch, 9, 6)
4453 scratch = 921 + correct3 ; drawObject(scratch, 13, 6)
4471 scratch = 913 + lethal1  ; drawObject(scratch, 24, 6)   ; right page
4489 scratch = 917 + lethal2  ; drawObject(scratch, 28, 6)
4507 scratch = 921 + lethal3  ; drawObject(scratch, 32, 6)
4547 print(254): pos(31,115) text="This configuration" / "is labelled `correct`."
      print(254):          text="This one is labelled" / "`certain death`."
```

So the twelve panels are really **three slots of four variants** — 913–916, 917–920, 921–924 — and the x-offsets (5/9/13 against 24/28/32 strips) put one arrangement on each page of the open book. `correct1–3` are VAR247–VAR249 and `lethal1–3` are VAR250–VAR252.



The twelve 24×48 object bitmaps from `90.LFL` at 6x. Read as three groups of four: each group supplies one slot on the page.

Both arrangements are rolled at boot

Room 76 — the boot room, which also carries `Insert Disk %c` and the opening scene at Barnett College — picks them during start-up:

```
13953 correct1 = getRandomNr(3)  correct2 = getRandomNr(3)  correct3 = getRandomNr(3)
13965 do lethal1 = getRandomNr(3) while (lethal1 == correct1)
13976 do lethal2 = getRandomNr(3) while (lethal2 == correct2)
13987 do lethal3 = getRandomNr(3) while (lethal3 == correct3)
```

The lethal arrangement is re-rolled until it differs from the correct one in *every* slot, so the two pages never share a figure in the same position.

What it is a hint for

Room 48 is a tunnel with a wooden door and three **statue** objects. The player sets each statue — `statue1-3`, `VAR127-129` — and the room tests the result against both pages of the diary:

```
16503  if (statue1==correct1 && statue2==correct2 && statue3==correct3)
        startScript(125,[4,5]) ; startSound(10)
        drawObject(643) ; drawObject(644)
        setClass(641,[0]) ; setState(641,1)          ; the way opens
16582  else if (statue1==lethal1 && statue2==lethal2 && statue3==lethal3)
        drawObject(660) ; drawObject(661)
        animateActor(...) ; actorOps(1): costume(52) ; death
```

Only those two arrangements do anything. Every other combination of the 64 possible is silently inert — you are not punished for experimenting, only for hitting the one arrangement the diary explicitly warns about.

Why this is not copy protection

It shares no machinery with room 92. Different objects (913–924 against 930/931), different variables, a different check in a different room. More decisively, the answer is *obtainable in the game*: the diary is an inventory item you pick up — “*It’s my father’s Grail Diary!*” — and reading it shows you the arrangement outright.

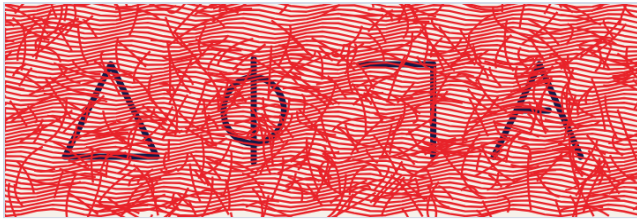
The randomisation is aimed at a different problem. Because both arrangements are drawn fresh each time the game starts, one player cannot tell another which statues to set; the answer has to come from the diary in that particular playthrough. That is an anti-walkthrough measure, not an anti-piracy one. The copy protection is room 92, and it needs a card that was never on the disk.

— THE MECHANISM

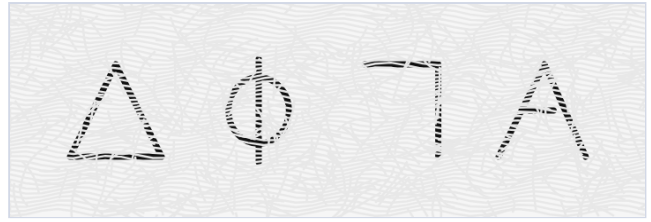
Why red plastic

Nothing on the disk describes the physical card — that part lives in the packaging, not the data. But the reason a publisher would print a lookup table in red is well understood, and it is the only way a paper key survives the photocopier.

The table is printed as dark ink buried under a dense field of red pattern. On white paper the two collide and the symbols are unreadable. Look through a red filter and the red pattern turns the same value as the paper and disappears, while the dark ink stays dark. A period photocopier, reading everything as luminance, flattened both to black — so the copy was worthless even when the disk copied perfectly.



As printed. Dark ink buried under a dense red pattern.



Through the red filter. The pattern drops to paper white; the ink stays dark.

Reconstruction of the printing principle, not a scan of the original card. The filtered view keeps only the red channel — what red acetate does to the light reaching your eye.

The same card, with and without the filter.

METHOD

Reproducing this

- **Pasti parsing.** Two details are easy to get wrong and both silently yield garbage: the track-image length is the size word itself (not size minus the header), and each track record is padded to an even length. Sector `dataOffset` values are relative to the *start of the track-image block*, not to the sector-data area — with that fixed, the last sector ends exactly on the record boundary, which is a free correctness check.
- **Room bitmaps.** A SCUMM v3 room header carries width at `+4`, height at `+6` and the image-block offset at `+10`. The image block opens with its own size word, then one `uint16` offset per 8-pixel strip, then EGA-encoded strip data.
- **Object bitmaps.** Objects use the same strip encoding but store no height. You can solve for it: the height is the value that makes each decoded strip consume exactly the distance to the next offset in the table. It came out unique every time.
- **Comparison opcodes.** SCUMM evaluates its conditionals as `operand2 OP variable`, not the other way round. Reading them the intuitive way inverts every branch in the derivation and produces an algorithm that looks plausible and is wrong.
- **Opcode 0xAE.** In SCUMM v3 this is a standalone `waitForMessage`, not the v5 `wait` that carries a subopcode byte. Consuming the extra byte desynchronises the rest of the script — which is what hid the third-strike branch from a first pass over this room. The tell that the one-byte reading is right: the block then decodes cleanly and lands exactly on the derivation's first instruction.

Analysis performed on the three `.stx` images only. Claims about the physical Translation Table card — that it was the red-filter document — follow from the on-disk evidence that the gate is a printed Section/Row/Column lookup, but the card itself is not part of the data and was not examined. Playing this today: ScummVM identifies the title as `indy3` and skips this check unless copy protection is explicitly enabled.